



**University of Tehran**  
College of Engineering  
School of Electrical & Computer Engineering

---

Experiment 4  
**Function Generator**

---

Digital Logic Laboratory  
ECE 045, ECE 895  
Laboratory Manual

Spring 1405



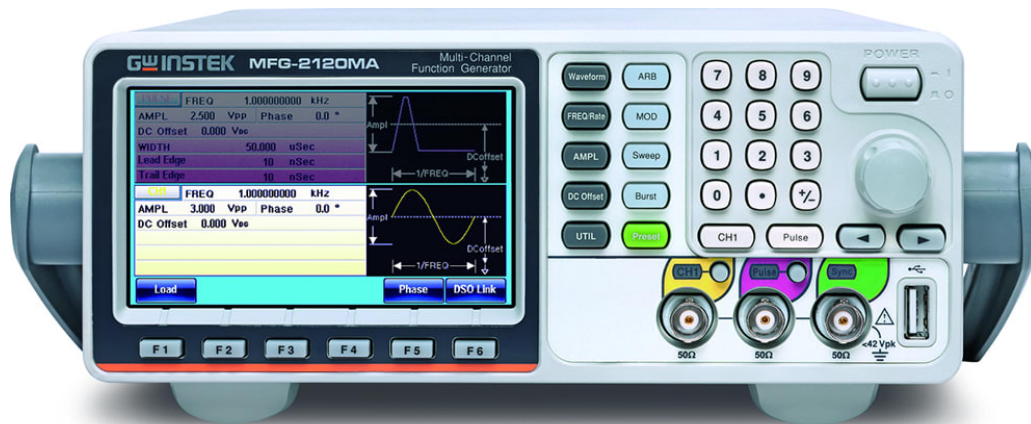
|       |                         |   |
|-------|-------------------------|---|
| DLLab | Function Generator..... | 1 |
|-------|-------------------------|---|

---

## Contents

|                      |    |
|----------------------|----|
| Contents             | 1  |
| Introduction         | 2  |
| 1 Waveform Generator | 3  |
| 2 Frequency Selector | 7  |
| 3 Amplitude Selector | 7  |
| 4 The Total design   | 8  |
| Acknowledgment       | 10 |

Figure 1: A single channel AFG from Instek company



## Introduction

An Arbitrary Function Generator (AFG) is an electronic test instrument that generates a wide variety of waveforms with different amplitude and frequency. Among them are sine, square, rhomboid, saw-tooth, and any arbitrary waveform. You can use AFGs to simply generate a series of basic test signals, replicate real-world signals, or create signals that are not otherwise available. These signals can then be used to learn more about how a circuit works, to characterize an electronic component, and to verify electronic theories. figure 1 shows a real AFG from Instek Company.

In this experiment, you are to design an Arbitrary Function Generator that is capable of generating each of the aforementioned waveforms with a wide range for frequency selection.

Figure 2 shows the simplified block diagram of an arbitrary generator. Based on these specifications there is a main component that generates one of the desired waveforms based on the function selectors' value, a frequency selector that sets the output signal frequency, and an amplitude selector. A ROM memory is usually embedded to store any other arbitrary waveform.

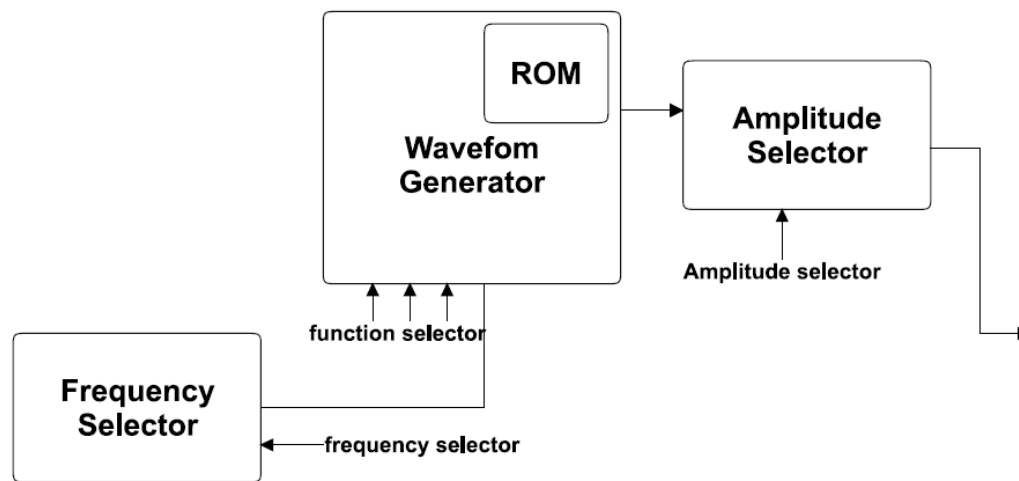
Accordingly, Below are the topics that are explained in the following experiment in detail:

- Waveform Generator
- Direct Digital Synthesis (DDS)
- Frequency Selector
- Amplitude Selector

By the end of this experiment, you should have learned:

- The principle of function generators
- Using ROM memories in your design
- Schematic design in Quartus II

Figure 2: A Block diagram of the Arbitrary Function Generator (AFG)



## 1 Waveform Generator

This module is the heart of this project. It produces desired functions. The output of this module is an 8-bit digital representing the amplitude of the signal. The supported functions, shown in figure 3, are sine, square, reciprocal, triangle, full-wave, and half-wave rectified signals.

- Waveforms square, reciprocal, and triangle are based on a counter that counts up or down with each clock for the period of the waveform. The output of the frequency selector is the input clock for this module that determines the discrete incremental values of this signal (resolution).
- Most modern function generators use Direct Digital Synthesis (DDS) for generating their output waveforms. DDS is used for generating arbitrary phase tunable output from a single fixed-frequency reference clock (like an oscillator). The output of the DDS module is a quantized version of the output files (usually a sinusoid). Figure 5 shows the hardware design of the DDS module. To generate the DDS signal, you should use a 1-port ROM memory to store the value of a sine wave for several clock cycles. This ROM module has 64 entries and only contains one quadrant of the sine wave. You will receive a file named `sine.mem` that is used for ROM initialization.

A phase accumulator module, PA, can generate the address location of this memory. The PA will enumerate all the data point positions of the entire period of the waveform. Two bits of the PA output denoted as "*signbit*" and "*phasepos*", are used to indicate the quadrant information. The next 6 bits of the PA output, denoted as "*addr*", are fed to the ROM address input. Figure 4 shows how  $2^{n+2}$  data points are generated from the  $2^n$  entries stored in the ROM.

Signed and magnitude numbers are used to represent the sinusoidal signal waveform. The

Figure 3: Different waveforms of function generator

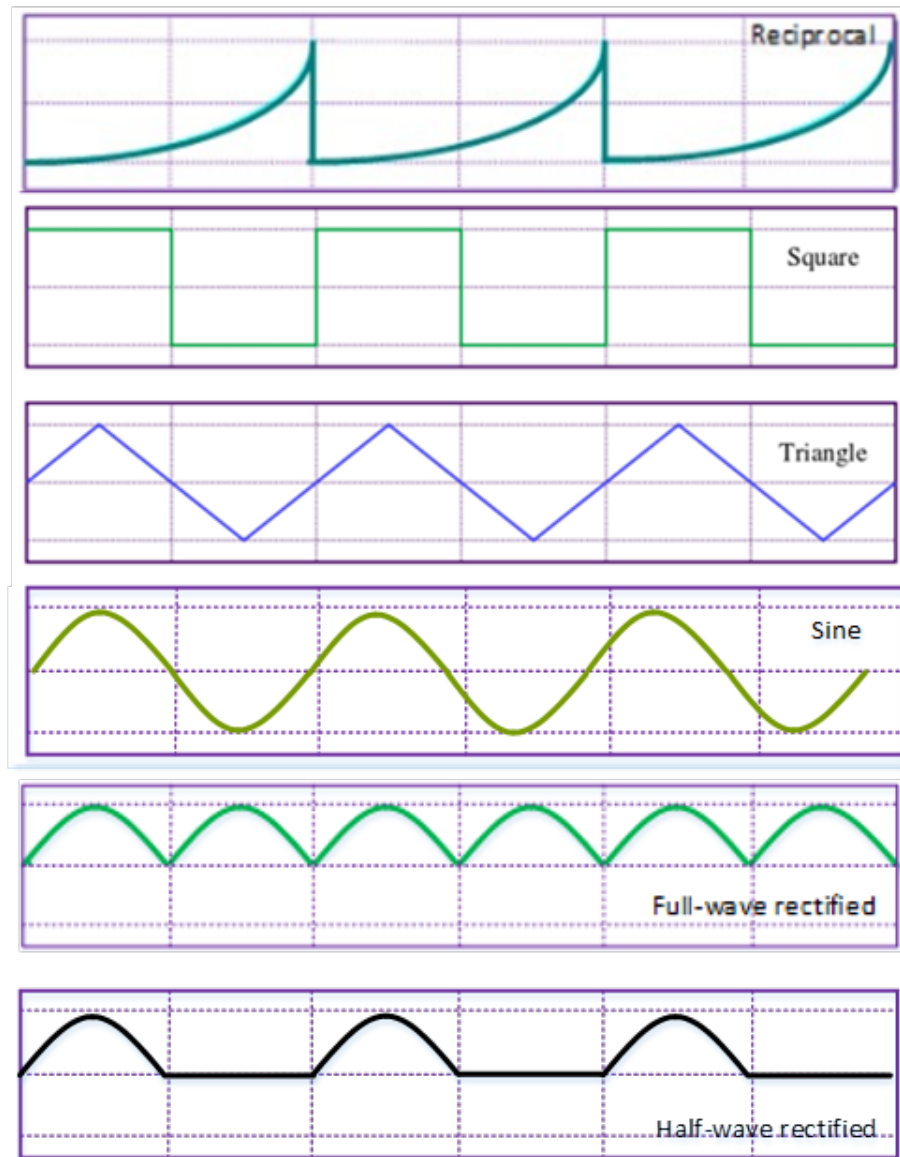
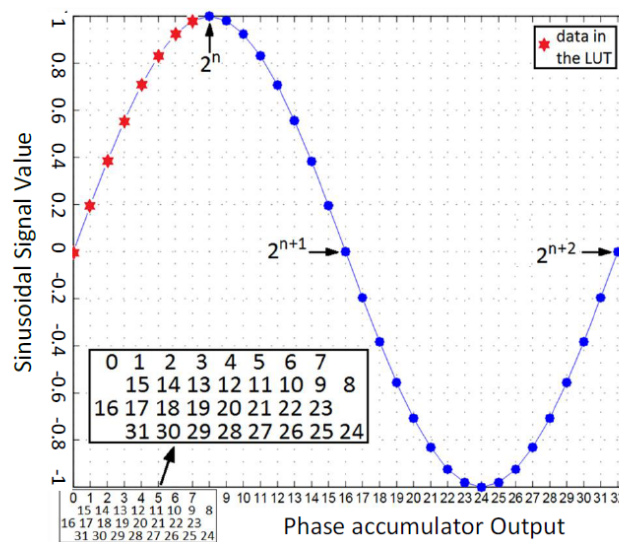


Figure 4: Phase accumulator output generation



"*phasepos*" bit of the PA output is used to select either "*addr*" or its 2's complementary code as the ROM address. The former case represents the first and third quadrants of the waveform and the latter corresponds to the second and the fourth quadrants. When "*phasepos* = 1" and "*addr* = 0", the corresponding data points are the positive and negative peaks of the sinusoidal waveform, whose magnitude value is not stored in the ROM. The all-zero-detection circuit and the following AND-gate are used to detect such cases. The output of the AND gate guides the output multiplexer to select either the ROM output or the digital code "11..11" as the signal magnitude accordingly.

The clock signal for DDS module should be a stable clock frequency. In this part, the 50-MHz clock frequency of FPGA is used but in the total design, this clock will be changed to the ring oscillator divided clock in the next sections.

Figure 5: DDS hardware design

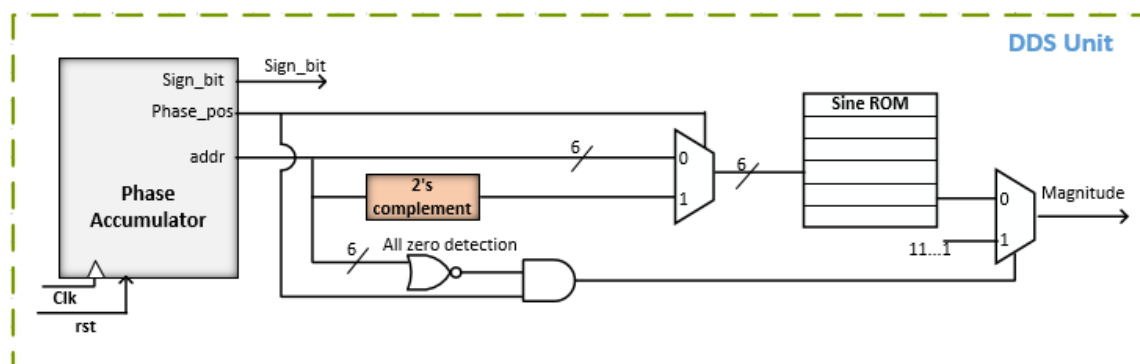


Figure 6: Block diagram of waveform generator with DDS

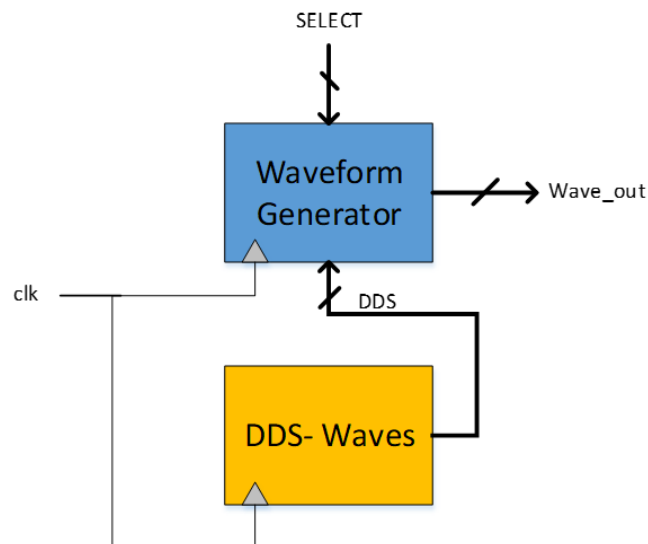


Table 1: Function selection

| func[2:0] | Function            |
|-----------|---------------------|
| 3'b000    | Reciprocal          |
| 3'b001    | Square              |
| 3'b010    | Triangle            |
| 3'b100    | DDS-Sine            |
| 3'b101    | Full-wave rectified |
| 3'b110    | Half-wave rectified |

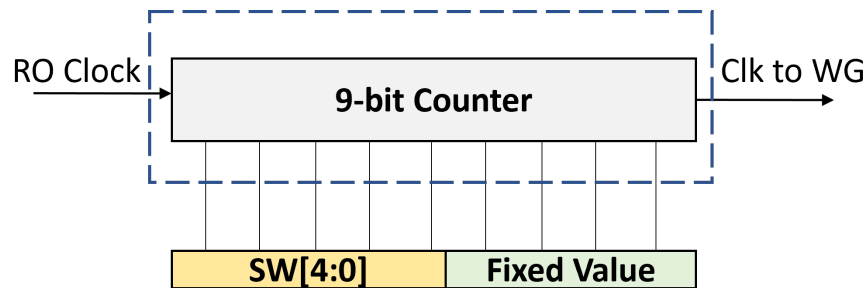
- The full-wave and half-wave rectified waveforms are both generated based on the sine wave.
- Now use the DDS sine wave and the counters in a way that generates waveforms similar to figure 3.

Figure 6 shows the block diagram of the waveform generator module with DDS.

Table 1 shows the order of function selection. These are the control signals for the waveform generator module.

1. Use the provided `sine.mem` file to initialize a 64-entry ROM for the DDS sine wave, implement a phase accumulator to generate the ROM address and quadrant bits, and generate the DDS sine wave from the single-quadrant ROM.
2. Write the Verilog code for the Waveform Generator module and use a multiplexer and other required gates, generate Reciprocal, Square, and Triangle waveforms from the counter, generate Full-wave and Half-wave rectified waveforms from the DDS sine wave, use a 3-bit input

Figure 7: Block diagram of frequency selector



func[2:0] to select the output waveform according to Table 1, and write a testbench with a 50 MHz clock to test all six waveforms.

- Before adding the frequency and amplitude selector, it is better to test your design in Modelsim. Therefore, write a simple testbench and verify the functionality of your design. For this part use the 50-MHz clock frequency. Include the Modelsim results for all the waveforms mentioned in Table 1.

## 2 Frequency Selector

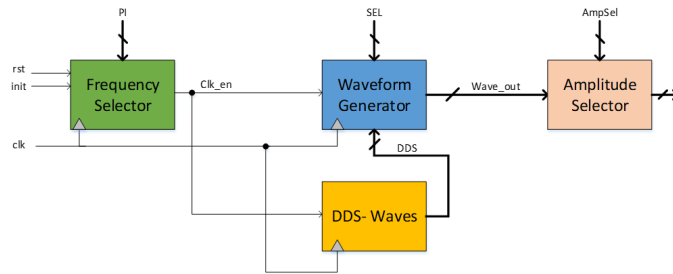
In order to set the frequency of the output signal a frequency selector is required. The frequency selector consists of a counter that divides a high source input signal to the desired value. For this purpose, you can write the Verilog description of a simple 9-bit counter like figure 7. For loading the division load value, you need to use an external load signal as init input.

- Set the 4 least significant bits of the counter to a fixed value in your code and set the remaining 5 bits as the input pins of the counter. Use PI[4:0] for parallel loads of the divider. Use init as an input for the frequency divider load.
- Modify the testbench of section one so that you can verify the design for different desired frequencies. To do this, change the parallel loads (PI[4:0]) for at least three different frequencies.
- Include the waveforms of each desired frequency in your report while mentioning the achieved frequencies, the input parallel loads, and expected frequencies.

## 3 Amplitude Selector

One option in the function generator is the amplitude of the generated wave. The task of this module is to scale down the amplitude of the waveforms. This can be done by dividing the output amplitude by a number. The value of the divisor is chosen by a 2-bit input. Dedicate two bits of input SW (SW[6:5]) to these selector inputs. The type of amplitude is selected by Ampsel[1:0] according to table 2.

Figure 8: Block diagram of the total design



1. Modify the testbench of section two so that you can verify the design for different amplitudes. To do this, change Ampsel[1:0] in your testbench. Figure 8 shows proper hardware design which you have to simulate before synthesis.
2. Include the waveforms of each desired amplitude in your report.

## 4 The Total design

When all the components are designed and verified, you can put them together in a schematic design. Quartus II has this capability to generate symbols for any HDL code with the corresponding inputs and outputs.

Finally, your top-level design must look like the block diagram of figure 9. The blocks shown in the red dotted area must be made into symbols in Quartus, then build the schematic like figure 10. After synthesis in Quartus, use the generated .vo file in ModelSim and instantiate it with the pre-synthesis DDS module, then prepare a testbench for this.

1. Click on *File* ▷ *New Project Wizard*.
2. Create a good directory for your project and complete the form as in the previous experiments.
3. In the *ADD/Remove Files in Project* page add all the Verilog code you have written.
4. By right-clicking on a Verilog code, you can create a symbol for it. If the symbol generation is done successfully, a file with the suffix *.bdf* will be generated.

Table 2: Amplitude selection

| SW[6:5] | Amplitude |
|---------|-----------|
| 2'b00   | 1         |
| 2'b01   | 2         |
| 2'b10   | 4         |
| 2'b11   | 8         |

Figure 9: Block diagram of the total design with synthesis boundary

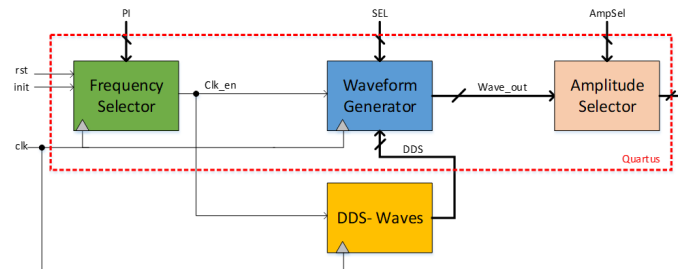
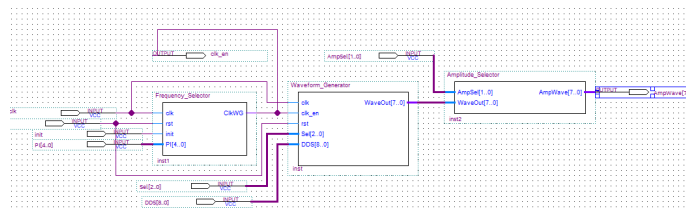
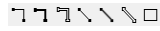


Figure 10: Final block diagram in Quartus II



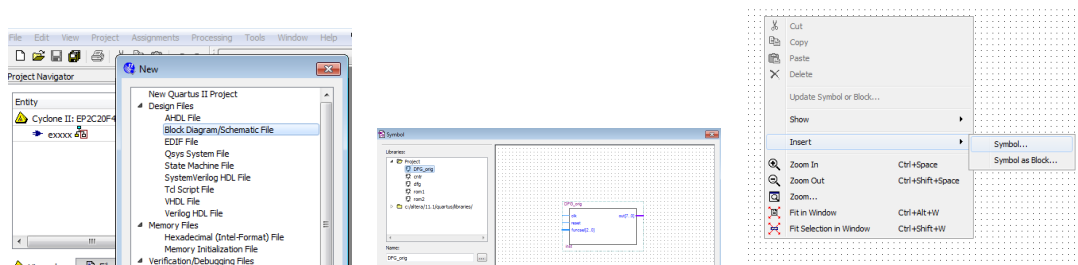
5. In ModelSim you can right-click on the output signal in waveform and choose the analog format to illustrate the proper waveform for you.

You should use bus or line tool from the top toolbar menu  and make all the interconnects between components.

Double-click on the blank space in the Graphic Editor window, or click on the icon in the toolbar that looks like an AND gate. A pop-up box will appear.

Expand the hierarchy in the Libraries box. First, expand libraries, then expand the library primitives, followed by expanding the library logic comprising the logic gates. Use the gates if any is needed. When the schematic design is completed, compile it as before. Program the design on FPGA and record all the results.

Figure 11: Quartus II



## Acknowledgment

This lab manual was prepared and developed by **Katayoon Basharkhah**, Ph.D. student of Digital Systems at the University of Tehran, under the supervision of Professor Zain Navabi.

This manual has been revised and edited by **Zahra Mahdavi** Ph.D. student of Digital Systems at the University of Tehran, **Iman Rasouli**, and **Zahra Hojati** undergraduate students of Electrical Engineering at the University of Tehran.